Eine Quenched-QCD Implementierung mittels Fortran-Coarrays

Nicolas Lang
Betreuer: Christian Hoelbling

25. Oktober 2017

Bergische Universität Wuppertal

QCD auf dem Gitter

Einführung: QCD auf dem Gitter

- Bietet Möglichkeit nicht-störungstheoretischer Berechnungen in QCD
- Regularisierung durch diskretisierte Raumzeit
- Gitterkonstante entspricht der Cut-off Energie im Impulsraum
- Niedrige Energieskalen zugänglich, bspw. Hadronenmassen

Das Pfadintegral

- Pfadintegralformalismus:

$$Z = \int \mathcal{D}\Phi e^{iS[\Phi]} \tag{1}$$

- Durch Einführung imaginärer Zeitkoordinate (Wick-Rotation):
  $\Rightarrow$ Minkowski-Metrik $\rightarrow$ euklidische Metrik
  $\Rightarrow$ komplexe Phase $\rightarrow$ Boltzmann-Faktor
  $\Rightarrow$ Verbindung von Quantenfeldtheorie und stat. Mechanik
- Diskretisierung der Raumzeit: 4-dimensionales Gitter
- Diskretisiertes Pfadintegral im euklidischen Raum:

$$Z = \int \mathcal{D}[\Phi] e^{-S_E[\Phi]} \tag{2}$$

Integrationsmaß:

$$\mathcal{D}[\Phi] = \prod_{n \in \Lambda} d\Phi(n) \tag{3}$$

QCD: Gluon Wirkung

- Hier: Beschränkung auf Eichfelder / keine Fermionen (quenched QCD)
- Einführung von Eich-Links zwischen den Gitterpunkten
- Gluon Wirkung:

$$S_G[U] = \frac{\beta}{3} \sum_{n \in \Lambda} \sum_{\mu < \nu} \Re \, \mathrm{tr}[1 - U_{\mu\nu}(n)] \tag{4}$$

- Plakette:

$$U_{\mu\nu}(n) = U_\mu(n) U_\nu(n + \hat{\mu}) U_\mu(n + \hat{\nu})^\dagger U_\nu(n)^\dagger \tag{5}$$

- Erwartungswerte von Operatoren:

$$\langle O \rangle = \frac{1}{Z} \int \mathcal{D}[U] e^{-S_G[U]} O[U] \tag{6}$$

- $4N$-dimensionales Integral $\Rightarrow$ keine exakte Berechnung möglich

Monte-Carlo Integration

- Approximation des Integrals durch "Stichprobenmittelwert" zufällig generierter Funktionswerte des Integranden
- Zufällige Konfigurationen sollten der Wahrscheinlichkeitsdichte

$$dP(U) = \frac{e^{-S[U]}\mathcal{D}[U]}{\int \mathcal{D}[U] e^{-S[U]}} \tag{7}$$

  zufolge generiert werden (Importance-Sampling)
- Erwartungswerte:

$$\langle O \rangle = \lim_{N\to\infty} \frac{1}{N} \sum_{n=1}^{N} O[U_n] \tag{8}$$

- Wie können solche Konfigurationen erzeugt werden?

Metropolis-Algorithmus

- Metropolis-Algorithmus: implementiert Markov-Kette zur iterativen Erzeugung der Konfigurationen durch Updates der alten Konfiguration
- Kurzformulierung:
  1. Starte bei beliebiger Konfiguration, wähle einen Link $U_\mu(n)$ und erzeuge Updatekandidaten $U_\mu(n)'$
  2. Berechne die Summe der *Staples*

$$A = \sum_{i=1}^{6} P_i = \sum_{\nu \neq \mu} \big(U_\nu(n+\hat{\mu}) U_\mu(n+\hat{\nu})^\dagger U_\nu(n)^\dagger \tag{9}$$

$$+ U_\nu(n+\hat{\mu}-\hat{\nu})^\dagger U_\mu(n-\hat{\nu})^\dagger U_\nu(n-\hat{\nu})\big) \tag{10}$$

     und damit die Wirkungsänderung

$$\Delta S = -\frac{\beta}{N} \Re \operatorname{tr}[(U_\mu(n)' - U_\mu(n))A] \tag{11}$$

  3. Akzeptiere Link-Update, falls $r \leq \exp(-\Delta S)$, wobei $r \in [0,1)$ Zufallszahl
  4. Wiederhole diese Schritte mit der neuen Konfiguration

Implementierung

Einschub: Fortran Coarrays

- Bestandteil des Fortran 2008-Standards
- Variablen mit *Codimension*
- *Images:* parallel ausgeführte Programminstanzen
- Beispiel: `real :: x(5)[*]`
  $*$ ist ein Platzhalter für die Anzahl der Images, wird zur Laufzeit festgelegt
- Unterstützung in GNU-Fortran 5 - für mehrere Images wird Bibliothek für Datenaustausch benötigt
- *OpenCoarrays*[1] ist eine solche
- Einschränkungen: u.a. keine Coarray-Pointer, keine Coarray-Returnvalues

[1] OpenCoarrays: Open-source Transport Layers Supporting Coarray Fortran Compilers

Konzept

- Ziele:
  - Implementierung der Quenched-QCD mit Coarrays
  - einfach zu benutzende API - globale "Feldperspektive"
- Umsetzung durch:
  - Typen (Klassen) für Felder und Feldvariablen die als Wrapper für (Co-)Arrays fungieren
  - überladene Operatoren für arithmetische Operationen
  - Select- und Shiftoperatoren für Felder, um Linkrichtung/-parität und Versatz festzulegen
  - Parallel ausgeführte Updates/Operationen auf ausgewählten Links
  - Günstige Speicherreihenfolge der Matrizen

Eichfeld-Typ

```
1  complex, allocatable :: &
2  data(:,:,:)[:]
3
4  type, public :: gauge_field
5      integer, allocatable :: indices
       (:,:,:,:,:)
6      logical :: tmp
7      integer :: dir
8      integer :: par
9      integer :: x_low, x_up, y_low,
       y_up, z_low, z_up, t_low, t_up
10     integer :: nx, ny, nz, nt
11 contains
12     ! ...
13     procedure :: initialize
14     procedure :: update
15     ! ...
16 end type gauge_field
```

- Globales data Coarray (unschön aber schwer zu vermeiden)
- *gauge_field*-Typ als Wrapper
- *dir* und *par* als Flags für Richtung und Parität
- Ausdehnung auf aktuellem Image
- *indices*-Array assoziiert euklidische- und Richtungsindizes mit (Co-)Arrayindizes
- Speicherreihenfolge (innen nach außen): x, y, z, t, par, dir

Feldvariablen-Typen

```
type, public :: su3_field_var
    complex, allocatable :: data
     (:,:,:)[:]
    logical :: conj = .false.
contains
    procedure :: add
    procedure :: add_glob_loc
    procedure :: assign
    procedure :: increment
    procedure :: real_trace
    procedure :: init_rnd
    procedure :: init_zero
    generic :: operator (+) => add,
     add_glob_loc
    generic :: assignment (=) =>
     assign
    generic :: operator (.rtrc.) =>
     real_trace
end type su3_field_var
```

- repräsentieren die Ergebnisse von Operatoren
- haben *data*-(Co-)Array als Member
- lokale "Version" geeignet für Temporaries - überladener Zuweisungsoperator
- kein Indexarray
- Reduzierungsfunktionen: *sum*, *mean*
- Verschiedene Datentypen:
  - SU(3)-Matrix-wertig
  - Reellwertig
  - Boolesch

Selectoperator

```
1  function select (a, b) result (tmp)
2      class(gauge_field), intent(in)
       :: a
3      integer, dimension(2), intent(in
       ) :: b
4      type(gauge_field) :: tmp
5
6      call a%copy_shift(tmp, 0, 0)
7      tmp%par = b(1)
8      tmp%dir = b(2)
9      tmp%tmp = .true.
10 end function
```

- Wählt eine Linkrichtung und Parität aus
- Akzeptiert *gauge_field* als ersten und integer array als zweiten Operanden
- Parität definiert als $p = (x+y+z+t) \ mod \ 2 \in \{0,1\}$
- $par = -1$ bedeutet alle sites
- $dir = 0$ bedeutet alle Richtungen

Shiftoperator

```
function shift (a, b) result (tmp)
    class(gauge_field), intent(in)
     :: a
    integer, dimension(2), intent(in
    ) :: b
    type(gauge_field) :: tmp

    call a%copy_shift(tmp, b(1), b
    (2))
    tmp%tmp = .true.
end function
```

- Verschiebt das Indexarray in die angeg. Richtung um den angeg. Betrag
- Akzeptiert *gauge_field* als ersten und integer array als zweiten Operanden

Arithmetische Operatoren und Vergleichsoperatoren

- Werden zwischen *gauge_field*- und Feldvariablentypen definiert
- Rückgabewert ist stets ein lokaler Feldvariablentyp
- Viele Typen $\Rightarrow$ hohe Anzahl möglicher Operatoren
- Funktionskörper immer ähnlich:
  - Iteration über alle euklidischen Indizes
  - Falls *dir* $= 0$: Iteration über alle Richtungen
  - Berechnung des x-Index unter Berücksicht. von Paritätsflag:

```
1 if (step_size == 2) then
2     xa = x-abs(mod(y+z+t,2)-a%par)
3 end if
4
```

- Interfaces: Skalare als Operanden und Überladung intrinsischer Funktionen (z.B. *exp*)

Arithmetische Operatoren: Multiply

```
1       res%data(:,:,idx) = multSu3( &
2       data( :,:,a%indices(0,xa,y,z,t,
        da)) &
3       [a%indices(1,xa,y,z,t,da)], &
4       (a%dir < 0), &
5       b%data(:,:,idx), &
6       b%conj )
7       idx = idx + 1
8
```

- Hier: Multiplikation zwischen *gauge_field* und *su3_field_var*
- Index und Coindex werden aus Indexarray gewonnen
- *multSu3* ist Wrapper um Basic Linear Algebra Subroutines (BLAS) Funktion cgemm
- Adjungierung bei neg. Wert von *dir* (Feld) bzw. *conj*-Flag (Feldvar.)

Eichfeld: Update

```
type(loc_su3_field_var), intent(inout) :: fv
type(loc_logical_field_var), intent(in) ::
    lfv
!... iterate over sites and directions
if (lfv%data(idx)) then
    if (fv%conj) then
        data(:,:,a%indices(0,la,k,j,i,da)) &
        [a%indices(1,la,k,j,i,da)] &
        = conjg(transpose(fv%data(:,:,idx)))
    else
        data(:,:,a%indices(0,la,k,j,i,da)) &
        [a%indices(1,la,k,j,i,da)] &
        = fv%data(:,:,idx)
    end if
    acc = acc + 1
end if
idx = idx + 1

```

- Iteriere über ausgewählte Links
- Führe Update durch falls *lfv%data(idx)* true ist
- Adjungiere bei entspr. Flag

Das Metropolismodul

Metropolismodul: Staple-Funktion

```
1     call staple%init_zero(s)
2     do i = 1, 4
3         if (i /= d) then
4             call staple%increment( &
5                 (u.sel.[p,i]) &
6                 * (u.shift.[i,1].sel.[p,d]) &
7                 * (u.shift.[d,1].sel.[p,-i]) &
8                 + (u.shift.[i,-1].sel.[p,-i]) &
9                 * (u.shift.[i,-1].sel.[p,d]) &
10                * (u.shift.[i,-1].shift.[d,1].
      sel.[p,i]) &
11                )
12        end if
13    end do
14
```

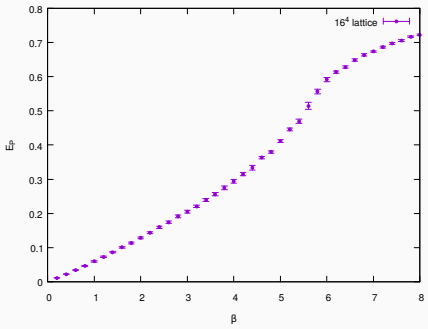
- Anwendung von Select und Shiftoperatoren
- Benutzt *increment*-Funktion, die für Feldvariablen definiert ist

Metropolismodul: Metro-Routine

```
1  do p = 1,2
2      do d = 1,4
3          stp = staple(u,p,d) ! calc. staple
4          stp%conj = .true.
5          call random_number(rnd1)     ! generate new link
6          call random_number(rnd2)
7          if (rnd1 >= 0.5) then
8              update_tab(ceiling(rnd2*100))%conj = .true.
9          else
10             update_tab(ceiling(rnd2*100))%conj = .false.
11         end if
12         new_link = (u.sel.[p,d]) * update_tab(ceiling(rnd2*100))
13         delta_s = -beta / 3.0 * .rtrc.((new_link - (u.sel.[p,d])) * stp)
14         call rnd_field%init_rnd(s)  ! accept with some prob.
15         call u%update(p, d, new_link, (rnd_field <= exp(-delta_s)),
       acc_count)
16         sync all
17     end do
18 end do
19
```

Messung

Messung der Plakettenwirkung



- Messung der Plakettenwirkung

$$E_P = \frac{\langle S_P \rangle}{6|\Lambda|} = \frac{1}{6|\Lambda| \cdot 3} \sum_{n \in \Lambda} \sum_{\mu < \nu} \Re \operatorname{tr}[U_{\mu\nu}] \tag{12}$$

- Erzeugt auf 16^4-Gitter
- Für jeden Schritt in β wurden 100 äquilibrierende Updates und 100 Gruppen von je zwei Updates und einer Messung durchgeführt
- Laufzeit von 4.41 Stunden auf Intel i7 2600k
- Gemessene Kurve entspricht dem erwarteten Bild